

Rebuild AST Prep from Zero

Everything needed to stand up a brand-new, fully working copy of the entire tool — GitHub, Supabase, and Vercel — from empty accounts to a live, sign-in-gated app, with every setting called out.

 **OWNER ONLY — not for distribution**

Read this once end-to-end before starting. The whole thing takes about 45–90 minutes the first time. Nothing here requires paid plans — the free tiers of GitHub, Supabase, and Vercel are enough. Dashboard menu labels shift occasionally; when a name has moved, look for the nearest equivalent described in the step.

Secrets rule. This document contains no passwords or keys, and neither should the repository. The only secret values you create (database password, Supabase `service_role` key) live in the Supabase dashboard and in Vercel's Environment Variables — never in a committed file. The app only ever uses the **anon public** key, which is safe to expose.

0 How the pieces fit together

Three services, connected in a loop:

Service	Role	Connects to
GitHub	Holds the source code (one repository, flat layout). You edit here.	Vercel watches it and rebuilds on every commit to <code>main</code> .
Vercel	Builds the code (Vite) and hosts the live website + PWA.	Reads two environment variables that point at Supabase.
Supabase	Accounts / sign-in (Auth) and the database (Postgres) with all rules and the sign-up gate.	Its Site URL / redirect URLs must list the Vercel domain so magic-link emails work.

The “glue” is exactly two values: `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`, set in Vercel, read by `supabaseClient.js`. Get those right and the loop closes.

1 GitHub — account & repository

Create the account (skip if you have one)

- 1 Go to `github.com` → **Sign up**. Use an email you control; verify it.
- 2 Pick the **Free** plan. No payment needed.

Create the repository

- 1 Top-right `+` → **New repository**.
- 2 **Repository name**: anything, e.g. `AST_training`. **Owner**: your account.
- 3 **Visibility**: Private is fine (Vercel can still read it once you authorize it). Public also works.
- 4 Leave “Add a README” unchecked (you'll upload files next). Click **Create repository**.

Flat layout — important. This project keeps all source files at the repository root (there is no `src/` folder), plus one `public/` folder for static assets served at the site root. So `index.html` references `/main.jsx`, not `/src/main.jsx`. Keep it that way — deploys break if files move into `src/`.

Upload the files

- 1 On the repo page: **Add file** → **Upload files** (works in mobile Safari too).
- 2 Drag in (or choose) every app file for the root (see the inventory in §10). To place assets in `public/`, either upload with the path `public/filename` in the name box, or create the folder first (**Add file** → **Create new file**, type `public/icon.svg`).
- 3 Set the commit to **Commit directly to the `main` branch** and commit.

Partial commits are the #1 gotcha. If you upload files in batches, make sure every changed file actually lands in the same commit. After a push you can verify a file is live at `raw.githubusercontent.com/<user>/<repo>/main/<file>`.

2 Supabase — create the project

- 1 Go to `supabase.com` → **Start your project / Sign in** (GitHub login is easiest).
- 2 **New project.** Choose an **Organization** (create one if prompted, Free plan).
- 3 **Name** the project (e.g. `ast-prep`). Choose a **Region** close to your users.
- 4 Set a strong **Database password** and **save it somewhere safe** — you rarely need it again, but it can't be recovered, only reset.
- 5 Create the project and wait ~2 minutes for it to provision.

3 Supabase — build the database

- 1 Left sidebar → **SQL Editor** → **New query**.
- 2 Paste the **entire contents** of `schema.sql` and click **Run**. It runs as one transaction — if anything errors, nothing is applied, so fix and re-run.
- 3 Then run any later migration files the same way (e.g. `sections.sql` for per-class section visibility). These are written to be idempotent (safe to re-run).

What `schema.sql` creates, so you know it worked:

- **Tables:** `profiles`, `app_settings`, `sessions` (classes), `memberships`, `runs`, `docs`, `leaderboard`, `question_overrides`.
- **Row-Level Security (RLS)** policies on every table — the real access control. The client gate is only cosmetic.
- **Functions:** `is_admin()`, `is_active()/am_i_active()`, `my_access()`, `my_sessions()`, the admin RPCs, and the two sign-up pieces below.
- **The sign-up gate** `hook_gate_signup(jsonb)` (validates the class join code) and the `handle_new_user()` trigger (creates the profile on sign-up).
- **Owner bootstrap:** `is_admin()` treats the owner email as admin automatically, and the schema flags that owner's profile as admin once it exists.

The owner email is baked in. Throughout the SQL and app, the super-admin is the email hard-coded as `sean.olesen@gmail.com`. If this instance should be owned by a different address, change it everywhere in S9 before running the SQL and deploying.

4 Supabase — Authentication settings

Under **Authentication** in the sidebar:

Setting (Authentication → ...)	Value & why
Sign In / Providers → Email	Enabled. This app uses email + password and magic links. Leave “Confirm email” on for production (users click a link to verify); turn off only for quick testing.
Allow new users to sign up	ON . Sign-ups are gated by the join-code hook, not by disabling sign-up. (Turning it off would block your learners.)
URL Configuration → Site URL	Your live Vercel URL, e.g. <code>https://your-app.vercel.app</code> . Magic-link emails redirect here (the app passes <code>window.location.origin</code>).
URL Configuration → Redirect URLs	Add the Vercel URL (and <code>http://localhost:5173</code> if you ever run it locally). Without this, magic-link sign-in fails with a redirect error.
Emails / SMTP	Supabase's built-in email works for low volume/testing. For reliable delivery to real learners, configure your own SMTP under Project Settings → Authentication → SMTP .

Enable the sign-up gate (the hook)

- 1 **Authentication** → **Hooks** (a.k.a. Auth Hooks).
- 2 Find **Before User Created** → choose **Postgres** as the type.
- 3 Schema `public`, function `hook_gate_signup`. Enable it.

The hook reads the join code the app sends in sign-up metadata (`event` → `user` → `user_metadata` → `join_code`). A valid, active code returns `{}` (allow); anything else returns an error (reject). The owner email is allowed through without a code, so you can create the first account. The schema already grants the hook to `supabase_auth_admin` and revokes it from everyone else — don't change those grants.

5 Supabase — get the API keys

- 1 **Project Settings** → **API** (or **Data API / API Keys** depending on dashboard version).
- 2 Copy the **Project URL** — this is `VITE_SUPABASE_URL`.
- 3 Copy the **anon public** key — this is `VITE_SUPABASE_ANON_KEY`.

Never use the `service_role` key in the app or the repo. It bypasses all RLS. The browser app must only ever use the **anon public** key. Keep `service_role` in the dashboard only.

6 Vercel — deploy the site

- 1 Go to `vercel.com` → sign up / log in with your GitHub account.
- 2 **Add New...** → **Project** → **Import** your GitHub repository (authorize Vercel to access it if asked).
- 3 **Framework Preset:** **Vite**. **Build command:** `vite build` (or `npm run build`). **Output directory:** `dist`. (Vercel usually detects these.)
- 4 **Environment Variables** — add both, for Production (and Preview):

Name	Value
<code>VITE_SUPABASE_URL</code>	the Project URL from §5
<code>VITE_SUPABASE_ANON_KEY</code>	the anon public key from §5

- 5 Click **Deploy**. When it finishes, note the live URL (e.g. `https://your-app.vercel.app`).

If you change env vars later, you must **redploy** for them to take effect (they're baked in at build time by Vite).

7 Close the loop

- 1 Copy the live Vercel URL from §6.
- 2 Back in **Supabase** → **Authentication** → **URL Configuration**, set **Site URL** to it and add it to **Redirect URLs** (§4).
- 3 If you added or changed anything in Vercel env vars, trigger a redeploy (Vercel → Deployments → ... → Redeploy).

8 Bootstrap the owner & first class

- 1 Open the live site. Choose **Sign up**. Enter the **owner email** (the one baked in, §9) and a password. You do **not** need a class code — the hook lets the owner through.
- 2 If email confirmation is on, click the link in the email, then sign in.
- 3 You should now see **Site Admin** on the Home screen (the owner is admin automatically via `is_admin()`). If it doesn't appear, re-run the owner-bootstrap lines of `schema.sql` (they flag the owner's profile as admin once the account exists).
- 4 In **Site Admin** → **Sessions & access**, create your first class. Copy its join code.
- 5 Test the learner flow: in a private browser window, **Sign up** with a different email + that join code, and confirm it works (and that a wrong code is rejected).

9 Make it *your* instance (new owner email)

To hand ownership to a different email, change it in **all four** places before running SQL / deploying, then keep them identical:

File	What to change
<code>Root.jsx</code>	the <code>SUPER_ADMIN</code> constant (near the top).
<code>SiteAnalytics.jsx</code>	the <code>SUPER_ADMIN</code> constant (near the top).
<code>schema.sql</code>	the email inside <code>is_admin()</code> , the owner-bootstrap <code>insert ... profiles</code> , and the allow-through check in <code>hook_gate_signup</code> .

Optionally also swap the app name, icons (in `public/`), and the manifest. If you change branding, regenerate the two learner-facing PDFs so they match.

10 File inventory & where things go

All app files live at the repo **root** except static assets, which go in **/public**. You don't need to memorize every file — upload whatever the working copy contains — but the shape is:

Location	Contents
/ (root)	<code>index.html</code> , <code>main.jsx</code> , <code>App.jsx</code> , <code>Root.jsx</code> , every <code>*App.jsx</code> tool, <code>Performance.jsx</code> , <code>SiteAnalytics.jsx</code> , <code>Leaderboard.jsx</code> , <code>glossary.jsx</code> , <code>i18n.jsx</code> / <code>i18n.fr.js</code> , <code>questions.js</code> / <code>questions2.js</code> , <code>storage.js</code> , <code>supabaseClient.js</code> , <code>overrides.js</code> , <code>vite.config.js</code> , <code>package.json</code> .
/public	Icons (<code>icon.svg</code> , <code>apple-touch-icon.png</code> , <code>favicon-32.png</code>), the web manifest, and the documentation PDFs served at the site root.
Run in Supabase	<code>schema.sql</code> first, then any migration <code>.sql</code> files (e.g. <code>sections.sql</code>). These are <u>not</u> part of the build — they're pasted into the SQL Editor.

11 Everyday operations

- **Ship a change:** upload/replace the file(s) on GitHub, commit to `main`. Vercel rebuilds automatically in ~1–2 minutes.
- **Database change:** paste the migration SQL into the Supabase SQL Editor and run it. Keep migrations idempotent.
- **Service-worker cache:** this is a PWA. After a deploy, hard-reload once or twice; for an installed (home-screen) app, fully close and reopen it so the new service worker takes over.
- **Static files (PDFs, etc.):** `vite.config.js` includes a `navigateFallbackDenylist` so opening a file URL isn't hijacked by the app shell. Keep it when editing the PWA config.
- **Verify a deploy:** compare the live page's build stamp, or check a file at `raw.githubusercontent.com/<user>/<repo>/main/<file>` to confirm the commit landed.

12 Troubleshooting

Symptom	Most likely cause & fix
Blank white page after deploy	Missing/incorrect env vars, or a file didn't commit. Check Vercel build logs; confirm both <code>VITE_</code> vars are set; redeploy.
Magic-link sign-in errors / redirects to the wrong place	The Vercel URL isn't in Supabase Site URL / Redirect URLs (§4, §7).
Sign-up always rejected	The Before User Created hook isn't enabled or isn't pointing at <code>hook_gate_signup</code> (§4); or the class code is inactive.
Owner can't see Site Admin	Owner email doesn't match in all places (§9); or re-run the owner-bootstrap SQL after the account exists (§8).
A change didn't appear	Partial commit (file never landed) or stale service worker. Verify via raw GitHub URL; hard-reload / reopen the PWA.
"Open" on a PDF shows the app	Missing <code>navigateFallbackDenylist</code> in <code>vite.config.js</code> (§11).
SQL error "cannot change return type"	A function's return type depends on a table you just altered — <code>drop function</code> it (by signature) before recreating.

AST Prep — Owner Rebuild Runbook. Keep this internal. It contains no secrets; ensure none are ever committed to the repository. The anon public key is safe to expose; the database password and `service_role` key are not.